

8_Dictionaries in Python Methods and Manipulations

Dictionaries are mutable and it is the combination of key, value pairs where key acts as an index and values can be any data type including data structures or a list, set, tuple or even a dictionary as well but key should be single name/any data type but not multiple items and it should be unique key in the defined dictionary. Dictionaries are defined in between curly braces just like sets.

```
dict={2}
print(type(dict ))
dict={2:1}
print(type(dict ))

[4] ✓ 0.0s

... <class 'set'>
... <class 'dict'>

▷ ▾
my_dict = {1: 'apple', 2: 'banana', 3: 'cherry'}
print(my_dict)
print(type(my_dict))
#empty dictionary
my_dict = {} #even set can be used to create an empty dictionary but by default {} considers as empty dictionary
print(my_dict)
print(type(my_dict))
myset=set() #remember {} is used to create an empty dictionary, not an empty set. To create an empty set, you need to use the set() function.
print(myset)
print(type(myset))

[13] ✓ 0.0s

... {1: 'apple', 2: 'banana', 3: 'cherry'}
... <class 'dict'>
... {}
... <class 'dict'>
... set()
... <class 'set'>

# dictionary values can be of any data type like list, tuple, set, etc.
my_dict = {"giveName": "Hanu", "age": 25, "hobbies": ["reading", "traveling", "coding"], "is_student": True}
print(my_dict)
print(len(my_dict))

[9] ✓ 0.0s

... {'giveName': 'Hanu', 'age': 25, 'hobbies': ['reading', 'traveling', 'coding'], 'is_student': True}
4

▷ ▾
# If you use the same key multiple times in a dictionary, the last value assigned to that key will overwrite any previous values.
# In other words, a dictionary cannot have duplicate keys. Each key must be unique within the dictionary.
# if you try to create a dictionary with duplicate keys, only the last value will be retained, and the previous values will be discarded.
# if you try to create a key with a mutable data type like a list or a set, it will raise a TypeError because mutable types are not hashable
# and cannot be used as dictionary keys.
dict_with_duplicate_keys = {1: 'apple', 2: 'banana', 1: 'cherry'}
print(dict_with_duplicate_keys) # Output: {1: 'cherry', 2: 'banana'}
dict_with_mutable_key = {[1, 2]: 'value'} # This will raise a TypeError
print(dict_with_mutable_key) # Output: TypeError: unhashable type: 'list'

[15] ✗ 0.0s

... {1: 'cherry', 2: 'banana'}

... -----
...
TypeError                                Traceback (most recent call last)
/tmp/ipykernel_27131/3753937489.py in <cell line: 0>()
      6 dict_with_duplicate_keys = {1: 'apple', 2: 'banana', 1: 'cherry'}
      7 print(dict_with_duplicate_keys) # Output: {1: 'cherry', 2: 'banana'}
----> 8 dict_with_mutable_key = {[1, 2]: 'value'} # This will raise a TypeError
      9 print(dict_with_mutable_key) # Output: TypeError: unhashable type: 'list'

TypeError: unhashable type: 'list'
```

```
# similar to nested lists, you can also have nested dictionaries, where a dictionary contains another dictionary as a value but not as a key.
nested_dict = {1: 'Apple', 2: {'fruit': 'Banana', 'color': 'Yellow'}, 3: 'Cherry'}
print(nested_dict)
```

[16] ✓ 0.0s

```
... {1: 'Apple', 2: {'fruit': 'Banana', 'color': 'Yellow'}, 3: 'Cherry'}
```

```
# Dictionary construction can also be done using the dict() constructor, which allows you to create a dictionary from
# a sequence of key-value pairs or from keyword arguments.
dict_from_pairs = dict(ename="Hanu", age=37, erole="Developer") # Using keyword arguments
print(dict_from_pairs)
```

[22] ✓ 0.0s

```
... {'ename': 'Hanu', 'age': 37, 'erole': 'Developer'}
```

```
# using zip function to create a dictionary from two lists
keys = ['name', 'age', 'city', 'colors']
values = ['Hanu', 25, 'New York', ["red", "green", "blue"]]
my_dict = dict(zip(keys, values))
print(my_dict) # Output: {'name': 'Hanu', 'age': 25, 'city': 'New York'}
```

[25] ✓ 0.0s

```
... {'name': 'Hanu', 'age': 25, 'city': 'New York', 'colors': ['red', 'green', 'blue']}
```

```
print(my_dict['name']) # Output: Hanu
print(my_dict['age']) # Output: 25
print(my_dict.get('age')) # Output: 25
# get() method is used to retrieve the value associated with a specified key in a dictionary but
# it does not raise an error if the key is not found. Instead, it returns None or a default value if specified.
```

[27] ✓ 0.0s

```
... Hanu
    25
    25
```

```
# dictionaries are mutable, meaning you can change their contents after creation.
# You can add new key-value pairs, update existing values, or remove items from a dictionary.
my_dict["name"] = "Rao"
my_dict["age"] = 37
print(my_dict) # Output: {'name': 'Rao', 'age': 37, 'city': 'New York'}
```

[37] ✓ 0.0s

```
... {'name': 'Rao', 'age': 37, 'city': 'New York', 'colors': ['red', 'green', 'blue']}
```

```
# remove an item from a dictionary using the del statement
del my_dict["age"] # using del statement to remove an item by key
my_dict.pop("city") # using pop() method to remove an item by key
print(my_dict) # Output: {'name': 'Rao'}
```

[38] ✓ 0.0s

```
... {'name': 'Rao', 'colors': ['red', 'green', 'blue']}
```

```
# we can assign pop() method's return value to a variable to store the removed value
removed_value = my_dict.pop("name") # Remove the item with key "name"
# but we cannot use del statement to store the removed value in a variable because del statement does not return any value.
# It simply removes the item from the dictionary.
print(removed_value) # Output: 'Rao'
```

[39] ✓ 0.0s

... Rao

```
# clearing a dictionary using clear() method
my_dict.clear() # Remove all items from the dictionary
print(my_dict) # Output: {}
```

[40] ✓ 0.0s

... {}

```
# keys() method returns a view object that displays a list of all the keys in the dictionary.
new_dict = {"name": "Hanu", "age": 25, "city": "New York"}
print(new_dict) # Output: {'name': 'Hanu', 'age': 25, 'city': 'New York'}
keys_view = new_dict.keys()
print(keys_view) # Output: dict_keys(['name', 'age', 'city'])
print(type(keys_view)) # Output: <class 'dict_keys'>
#values() method returns a view object that displays a list of all the values in the dictionary.
values_view = new_dict.values()
print(values_view) # Output: dict_values(['Hanu', 25, 'New York'])
print(type(values_view)) # Output: <class 'dict_values'>
#items() method returns a view object that displays a list of all the key-value pairs in
items_view = new_dict.items()
print(items_view) # Output: dict_items([('name', 'Hanu'), ('age', 25), ('city', 'New York')])
print(type(items_view)) # Output: <class 'dict_items'>
```

[43] ✓ 0.0s

```
... {'name': 'Hanu', 'age': 25, 'city': 'New York'}
dict_keys(['name', 'age', 'city'])
<class 'dict_keys'>
dict_values(['Hanu', 25, 'New York'])
<class 'dict_values'>
dict_items([('name', 'Hanu'), ('age', 25), ('city', 'New York')])
<class 'dict_items'>
```

```
# update method is used to update the contents of a dictionary with key-value pairs from another dictionary or an iterable of key-value pairs.
new_dict1 = {"role": "Developer", "language": "Python"}
new_dict2 = {"experience": 2, "skills": ["Python", "JavaScript"]}

new_dict1.update(new_dict2)
print(new_dict1) # Output: {'role': 'Developer', 'language': 'Python', 'experience': 2, 'skills': ['Python', 'JavaScript']}
new_dict1.update({"City_Name": "Plano", "State": "TX"})
print(new_dict1) # Output: {'role': 'Developer', 'language': 'Python', 'experience': 2, 'skills': ['Python', 'JavaScript'], 'City_Name': 'Plano', 'State': 'TX'}
```

[45] ✓ 0.0s

```
... {'role': 'Developer', 'language': 'Python', 'experience': 2, 'skills': ['Python', 'JavaScript']}
{'role': 'Developer', 'language': 'Python', 'experience': 2, 'skills': ['Python', 'JavaScript'], 'City_Name': 'Plano', 'State': 'TX'}
```